



ESCUELA COLOMBIANA DE INGENIERIA

BIENVENIDOS ESTUDIANTES

DOSW 1

DOSW NO ES UN CURSO TRADICIONAL...

DOSW es una fábrica de software cuyo objetivo es buscar a los mejores talentos del mercado estudiantil, personas con ganas de aprender, mejorar y construir software de verdad



Durante el semestre trabajaremos como un equipo de desarrollo real, con retos técnicos, decisiones de arquitectura, trabajo en equipo, presión de entregas y el tiempo contado



QUIEN LIDERA ESTA EMPRESA?

ANDRES MARTIN CANTOR URREGO



PROFESOR

27 AÑOS



Rapyd
PayU



ANDRES.CANTOR-U@ESCUELAING.EDU.CO



ABRIMOS VACANTES

JUNIOR SOFTWARE ENGINEER

Modalidad: Academica – Simulacion de un entorno real

Duracion: Semestre academico 2026-2

Requisitos

:

- Conocimientos básicos de programación orientada a eventos
- Conocimientos básicos de Bases de datos
- Haber usado al menos un lenguaje de programación
- Interés por aprender frameworks, herramientas y buenas prácticas
- Capacidad de trabajo en equipo
- Disposición para salir de la zona de confort

Este no es un proceso para “medir quién sabe más”, sino para identificar potencial y crecimiento.



COMO FUNCIONARA EL SEMESTRE



**ONBOARDING –
HIRING**

SEMANA 1



BOOTCAMP

SEMANA 2 A LA 5



**ENGINEERING
ASSIGNMENT**

SEMANA 6 A LA 11



**UNIFIED ENTERPRISE
SOLUTION**

SEMANA 12 A LA 17



A NIVEL DEL CRONOGRAMA DE LA ESCUELA...



**ONBOARDING -
HIRING**



BOOTCAMP

CORTE 1

Sistemas de Control de Version
Programacion Funcional
Patrones de diseño
Principios de diseño y diagramacion
Pruebas
Metodologias agiles



ENGINEERING ASSISGNMENT

CORTE 2

Arquitectura Web
Persistencia
Integracion Continua y Despliegue
continuo
Microcontenedores y Docker
FrontEnd y UX
Security



**UNIFIED ENTERPRISE
SOLUTION**

CORTE 3

Construccion de una solucion
requerida por una entidad real
Modulo de Profundizacion - Principios
de Arquitectura
Presentacion y Pitch



COMO VAN A SER EVALUADOS?

CORTE 1

50% Parcial (teorico + practico)
20% Laboratorios de Bootcamp
20% Bitacora
10% Quices o Talleres



**TALENT
READINESS**

CORTE 2

50% Parcial (teorico + practico)
40% Proyecto Engineering Assignment
10% Quices o Talleres



**SKILL
ACCELERATION
PHASE**

CORTE 3

80% Proyecto Final
(Nota Grupal y del Curso)
20% Trabajo en Clase



**PROBATION
PERIOD**

**TODOS LOS CORTES TENDRAN LA OPCION DE
BONIFICACION**



ENGINEERING READINESS PROGRAM

TALENT READINESS

Diseñado en caso de que no pases la primera semana de Onboarding

SKILL ACCELERATION PHASE

Diseñado para aquellos estudiantes que no pasen el bootcamp

PROBATION PERIOD

Diseñado para aquellos estudiantes que no pasen el engineering assignment

PROGRAMA INTENSIVO PARA EL REFUERZO DE LAS HABILIDADES DENTRO DEL CURSO, SIRVE PARA CORREGIR BRECHAS Y VOLVER A ESTAR AL DIA





CONOZCAMOS A LOS ASPIRANTES

Hora de la dinamica de la presentacion inicial

Cada estudiante debera Presentarse: Su nombre completo (nombre favorito o apodo), Edad, Semestre actual, cual es su motivacion, cual es su mayor fortaleza y que han escuchado del curso

Cada uno debe entregar su Hoja de vida





REGLAS DE LAS SESIONES

1.LA ASISTENCIA ES OBLIGATORIA, SI TIENEN ALGUN PROBLEMA JUSTIFICABLE NO DUDEN EN ESCRIBIRME PARA VALES LA INASISTENCIA

1.INGRESO DE ESTUDIANTES EXTERNOS - QUIZ SORPRESA Y NOTA PARA TODO EL CURSO

1.EL PLAGIO NO ESTA PERMITIDO - CUALQUIER INTENTO DE COPIA DE ALGUNO AFECTARA A TODO EL CURSO

RECOMENDACION: USEN LA I.A COMO MOTOR DE CONOCIMIENTO NO SU ESCLAVO DIGITAL



DOSW COMPANY

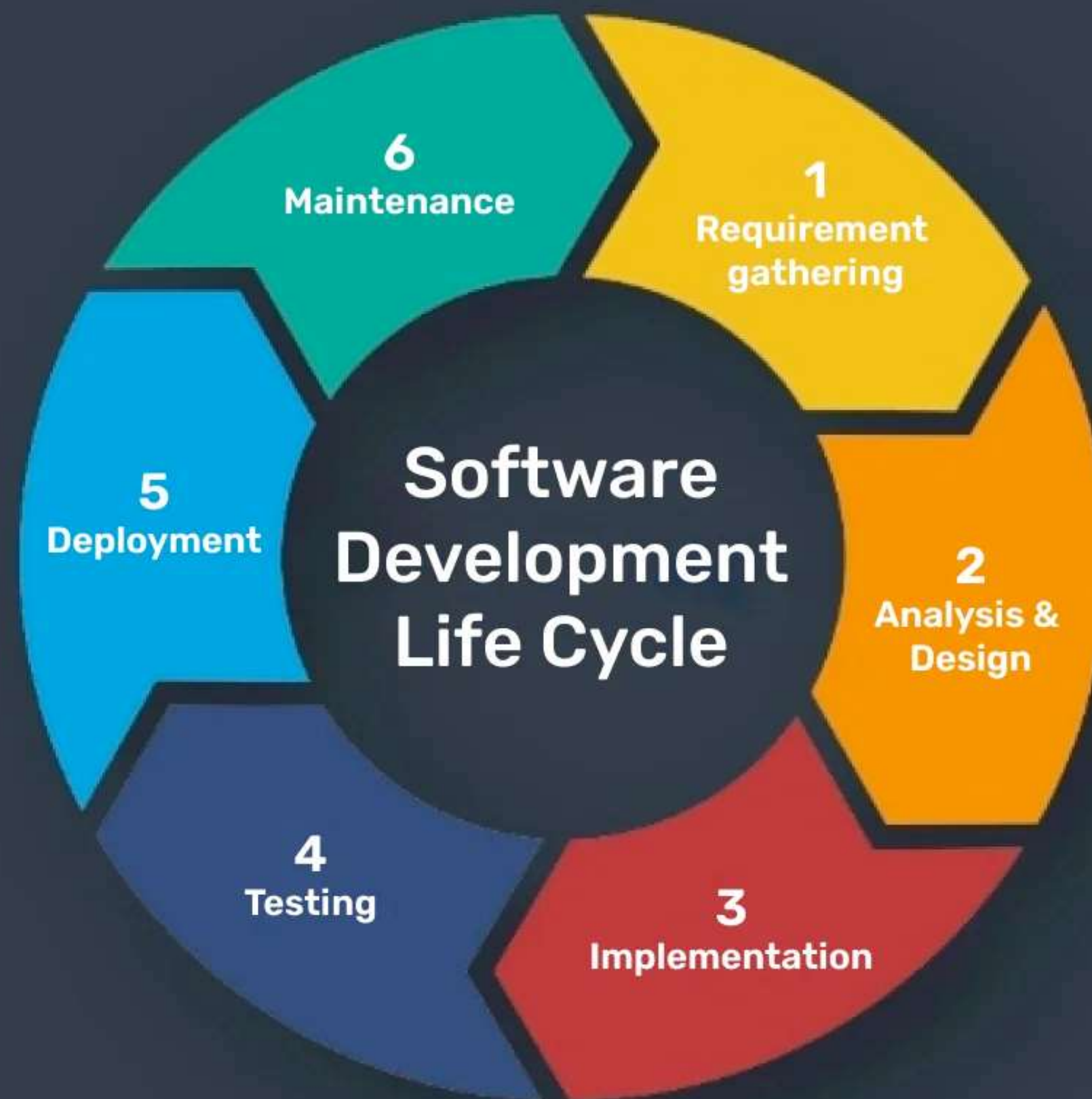
BIENVENIDOS A
SU ONBOARDING





DOSW COMPANY

PORQUE ES IMPORTANTE EL CICLO DE VIDA DEL SOFTWARE?





PRIMER DESAFIO

EL FRANKENSTEIN

4 Grupos de 4 personas

1 Grupos de 5 personas

Reunanse en el equipo que les toco

Analicen como van a organizarse (Receptor inicial,
Transmisores, Implementador)

Cuando esten listos llaman al profesor para darles su
numero de prioridad y su nombre de equipo





Se llama solo al receptor inicial de cada fila, la persona puede Escuchar solo 1 vez la descripción del personaje.

Cada participante debe tener los ojos tapados.

El receptor pasa la descripción una sola vez al siguiente emisor sin repetir.

El dibujante se quita la venda y dibuja con lo que recibe

El equipo tiene 3 intentos para adivinar el personaje con las pistas que pueden recopilar cumpliendo un reto, si fallan otra fila puede levantar la mano y robar el punto (solo tienen 1 intento)

¿Preparados?





PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 1

ENUNCIADO

Con la risa escondía lo que el alma sentía,
las palabras veloces eran su compañía.
Entre amigos halló un lugar para crecer,
y quien menos esperaba lo vio cambiar por querer.
No fue el más fuerte, ni el más popular,
pero con una broma te podía desarmar.
Si buscas mi esencia, no mi profesión,
en cada respuesta encontrarás que vivi en new
york.
¿Quién soy?

PISTAS

Su mayor evolución ocurre cuando deja de huir del
compromiso.
Es recordado más por su forma de hablar que por lo que hacía para
vivir.
Su historia demuestra que el humor puede ser un mecanismo para
ocultar inseguridades.



DOSW COMPANY

PRIMER DESAFIO EL FRANKENSTEIN

PERSONAJE 1



CHANDLER BING



PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 2

ENUNCIADO

Nací para sembrar el mayor temor,
pero terminé enseñando el verdadero valor.
Mi destino parecía imposible de cambiar,
aunque cada lección invitaba a no dejar de
luchar.

Muchos me persiguen con un mismo ideal,
y aun así los preparo para su prueba final.
No busco venganza ni imponer mi poder,
mi mayor victoria fue verlos crecer.
¿Quién soy?

PISTAS

Su mayor enemigo no era un héroe, sino el
tiempo.

La velocidad es una de sus habilidades más famosas

Su historia gira alrededor de un salón de clases donde el aprendizaje era
tan importante como la misión.



DOSW COMPANY

PRIMER DESAFIO EL FRANKENSTEIN

PERSONAJE 2



KURO SENSEI



PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 3

ENUNCIADO

No nací entre lujos ni con fama al llegar,
cada paso fue esfuerzo para poder brillar.
El tiempo fue rival, la disciplina mi voz,
y donde muchos dudaban, respondía con
decisión.

Me persigue el ruido de eterna comparación,
pero escribí mi historia alejado de vos.
Caí muchas veces, volví a despegar,
porque los récords existen para intentar superar.
¿Quién soy?

PISTAS

Ha conseguido títulos en las principales ligas de más de un país europeo.
Su celebración más famosa es reconocida incluso por personas que no siguen su deporte.

Tiene un índice de grasa bastante bajo gracias al ejercicio constante



DOSW COMPANY

PRIMER DESAFIO EL FRANKENSTEIN

PERSONAJE 3



CRISTIANO RONALDO



PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 4

ENUNCIADO

Nací entre secretos que nunca elegí,
y crecí persiguiendo un lugar para mí.
A veces fui sombra, otras, resplandor,
cambiando de rostro según la ocasión.

Dicen que engaño, que no soy de fiar,
pero hasta de mi mismo me pude enamorar.

Ni héroe completo, ni villano sin fin,
mi historia demuestra que el destino no siempre es el inicio
del árbol.
¿Quién soy?

PISTAS

Su identidad está marcada por un origen que descubre mucho
después de haber crecido.

El engaño y las ilusiones son herramientas, pero no representan
el conflicto más importante de su historia.

Su evolución lo lleva a comprender que el poder no siempre
consiste en gobernar, sino en asumir un propósito mucho mas
glorioso.



DOSW COMPANY

PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 4



LOKI



PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 5

ENUNCIADO

Soñé con la gloria siguiendo el honor,
creí que un héroe bastaba en el fútbol feroz.

La derrota cambió aquello que fui,
y un nuevo camino apareció frente a mí.
Mi fuerza intimida, mi voluntad también,
pero el precio del cambio no lo entiende cualquiera.
Entre sueños y orgullo elegí continuar,
aunque ya no fuera el mismo al volver a jugar.
¿Quién soy?

PISTAS

Su forma de entender el fútbol cambia radicalmente después
de una eliminación que no fue el final de su historia.

Destaca por un físico sobresaliente y un potente disparo con su
pierna dominante.

Su regreso sorprende porque vuelve con una personalidad muy
distinta a la que tenía al inicio del proyecto



DOSW COMPANY

PRIMER DESAFIO EL FRANKENSTEIN

PERSONAJE 5



KUNIGAMI



PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 6

ENUNCIADO

Entre pasillos donde el lujo es la ley,
siempre busqué ser la reina sin llevar corona de
rey.

Las notas perfectas, el porte ideal,
hacían creer que nada me podía afectar.
Mi lengua era filo, mi mirada poder,
y pocos lograban hacerme ceder.

Detrás del orgullo y de tanta ambición,
también escondía miedo y un gran corazón.
Cuando todo parecía imposible sostener,
descubrí que ganar no siempre es vencer.
¿Quién soy?

PISTAS

Su uniforme escolar es tan representativo como su
personalidad dominante.

Durante gran parte de la historia compite por ser la mejor en
todo, tanto académica como socialmente.

Es uno de los personajes cuya evolución pasa de la arrogancia a
mostrar una faceta mucho más humana y vulnerable.



DOSW COMPANY

PRIMER DESAFIO EL FRANKENSTEIN

PERSONAJE 6



LUCRECIA



PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 7

ENUNCIADO

La suerte me esquivó durante años sin fin,
pero seguí apostando esperando el porvenir.
Con deudas al hombro y el alma cansada,
acepté una invitación tan extraña como inesperada.
Donde jugar era también sobrevivir,
aprendí que no todos querían compartir.
Perdí más de lo que llegué a ganar,
y aun con el premio, no pude celebrar.
Mi historia no termina cuando logro vencer,
porque hay juegos que nadie debería volver a ver.
¿Quién soy?

PISTAS

Su número dentro de la competencia terminó convirtiéndose
en uno de los más reconocidos de la serie.

Antes de participar, era conocido por tomar malas decisiones
financieras y depender de las apuestas.

La victoria no representó el final de su conflicto; fue el
comienzo de un propósito mucho mayor.



DOSW COMPANY

PRIMER DESAFIO EL FRANKENSTEIN

PERSONAJE 7



SEONG GI HUN - 456



PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 8

ENUNCIADO

Nací donde elegir marcaba el destino,
pero mi esencia nunca siguió un solo camino.
Mientras todos encajaban en un mismo lugar,
yo llevaba un secreto difícil de ocultar.
Salté antes que el miedo pudiera decidir,
y descubrí que el valor también es resistir.
No luché por una sola bandera o color,
sino por un mundo libre del miedo y del control.
Lo que me hacía diferente fue también mi condena,
pero nunca permití que definiera mi cadena.
¿Quién soy?

PISTAS

Su mayor fortaleza era que no podía ser definida por una sola cualidad.

Una prueba diseñada para controlar a las personas resultaba mucho menos efectiva en ella que en los demás.

Su historia comienza con una elección que cambia por completo su vida y desafía el sistema que gobernaba su sociedad.



DOSW COMPANY

PRIMER DESAFIO EL FRANKENSTEIN

PERSONAJE 8



TRIS PRIOR



PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 9

ENUNCIADO

No empuño la fuerza como mi mayor poder,
prefiero pensar antes de responder.
Los mares son mapas escritos para mí,
y pocos comprenden lo que puedo descubrir.
El pasado dejó cicatrices difíciles de borrar,
pero nunca apagó mis ganas de navegar.
Persigo un sueño que parece no tener final,
dibujar un mundo que nadie conoce igual.
Entre tormentas, tesoros y libertad,
encontré una familia sin compartir sangre de
verdad.
¿Quién soy?

PISTAS

Su mayor sueño no consiste en encontrar un tesoro, sino en
completar una obra que abarque todo el mundo.

Aunque no destaca por su fuerza física, es una pieza
indispensable para que su tripulación sobreviva en el mar.

Su infancia estuvo marcada por el dominio de unos piratas que
la obligaron a desarrollar un talento muy específico desde muy
joven.



DOSW COMPANY

PRIMER DESAFIO EL FRANKENSTEIN

PERSONAJE 9



NAMI



PRIMER DESAFIO

EL FRANKENSTEIN

PERSONAJE 10

ENUNCIADO

Mi cabello ondulado viaja antes que mi andar,

y por la banda nunca dejo de luchar.

No soy quien más goles suele marcar, pero una paella me acabo de almorzar.

Defiendo, ataco, vuelvo a empezar, pero aveces me arrastran como trapeador.

Muchos me recuerdan por la gran final, donde el esfuerzo pesó más que la estrella.

Entre centros, barridas y velocidad, mi mayor virtud siempre fue la intensidad.

¿Quién soy?

PISTAS

Su aspecto físico lo hace reconocible incluso antes de tocar el balón.

Su posición principal está en una de las bandas, aunque puede desempeñar varios roles defensivos.

Uno de los momentos más recordados de su carrera ocurrió representando a su selección en un gran torneo internacional, donde fue protagonista durante el camino al título



DOSW COMPANY

PRIMER DESAFIO EL FRANKENSTEIN

PERSONAJE 10



MARC CUCURELLA



PRIMER DESAFIO

GANADORES

RECOMPENZA: PODERES PARA LA
SIGUIENTE CLASE Y
EXONERACION DE LOS QUICES
DEL CORTE





DOSW COMPANY

REFLEXION

DEL EJERCICIO

¿Porque los dibujos se desviaron tanto del original?

¿Que pudo haber fallado en el proceso?

¿Que podriamos haber hecho diferente?

¿Porque creen que es importante la planificacion antes de cualquier actividad?

¿Porque creen que esta actividad se relaciona con DOSW?



DOSW COMPANY

REFLEXION

DEL EJERCICIO



How the Customer Explained it



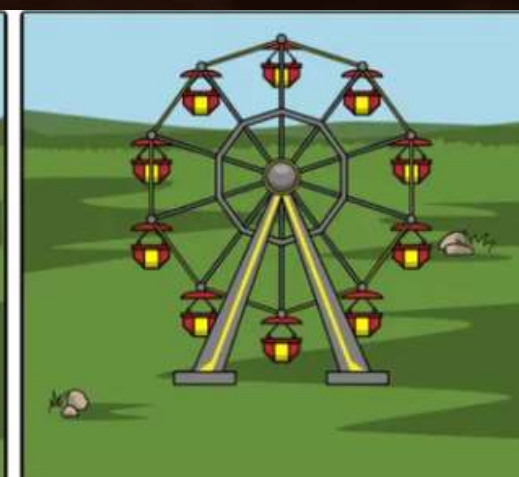
How the Engineer Designed it



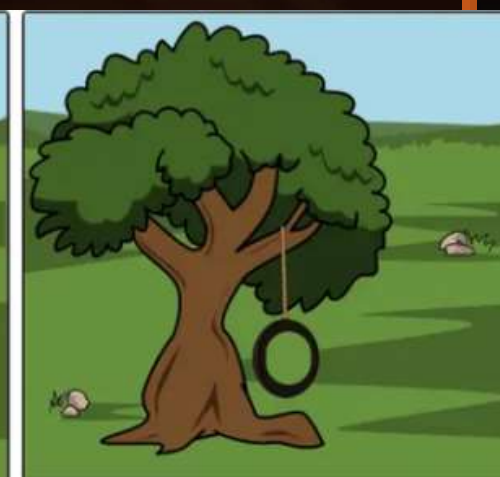
How the sales executive described it



How the Project was Documented



How the Customer was Billed



What the Customer Really Wanted



DOSW COMPANY



ACEPTAN ESTE
NUEVO RETO?



DOSW COMPANY

COMPLETED

PRIMERA FASE

LET'S TAKE A PIC



SU NUEVO MEJOR AMIGO

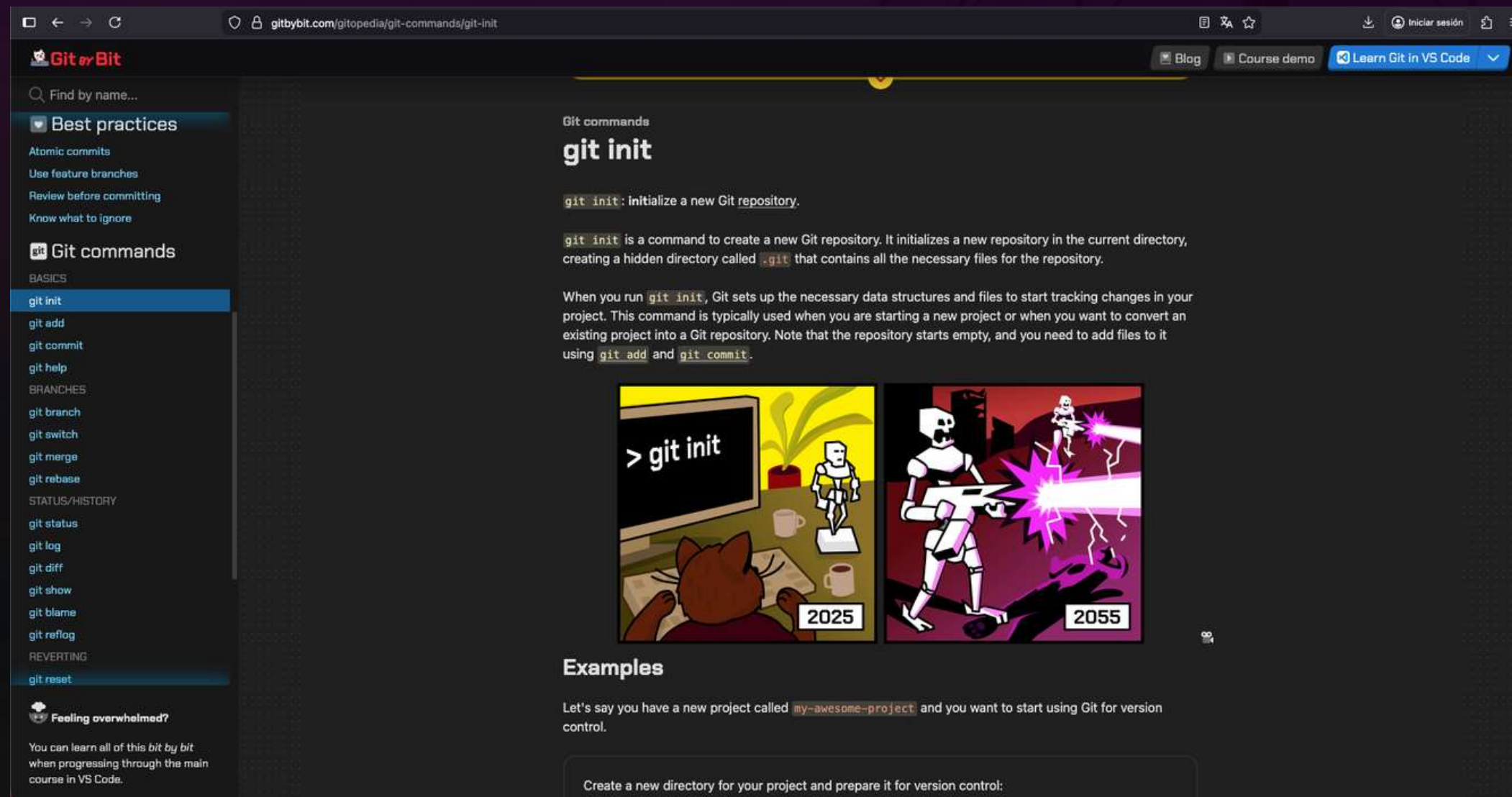
Name
120525_document_updated.txt
120604_document.txt
120605_document_amended.txt
120605_document_John.txt
120605_document_latest.txt
120605_document_latestcopy.txt
120605_document.txt
120602_document.txt
document_meeting.txt



GIT

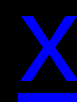


Software de control de versiones, pensado para la eficiencia, confiabilidad y compatibilidad del mantenimiento de versiones de aplicaciones cuando estas tienen un gran número de archivos de código fuente



GITOPEDIA

<https://gitbybit.com/gitopedia/index>





GIT FLOW

Git plantea una gran libertad en la forma de trabajar en torno a un proyecto. Sin embargo, para coordinar el trabajo de un grupo de personas en torno a un proyecto es necesario acordar como se va a trabajar con Git. A estos acuerdos se les llama flujo de trabajo.

Es el flujo de trabajo más conocido. Está pensado para aquellos proyectos que tienen entregables y ciclos de desarrollo bien definidos. Está basado en dos grandes ramas con infinito tiempo de vida (ramas master y develop) y varias ramas de apoyo, unas orientadas al desarrollo de nuevas funcionalidades (ramas feature-*), otras al arreglo de errores (hotfix-*) y otras orientadas a la preparación de nuevas versiones de producción (ramas release-*).

La herramienta gitflow facilita la automatización de las tareas implicadas en este flujo de trabajo

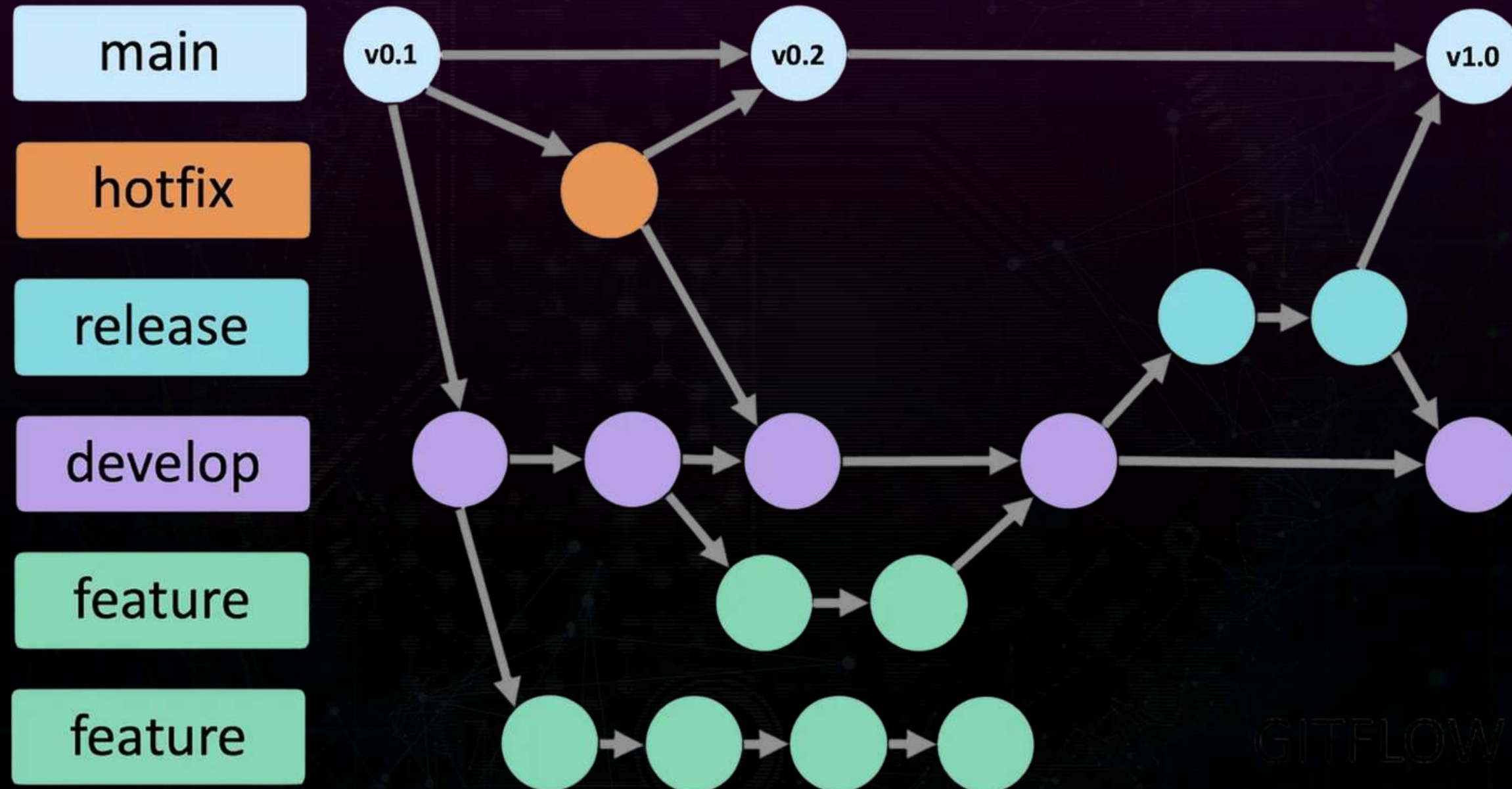
Master - La rama principal
Develop - Sacada de master, es la rama de la integración, las nuevas funcionalidades se integran a ella
Features - Cada nueva funcionalidad con una rama específica
Hotfix - Errores en producción deben ser arreglados urgentemente
Release - Apoyan la preparación a nuevas versiones de producción



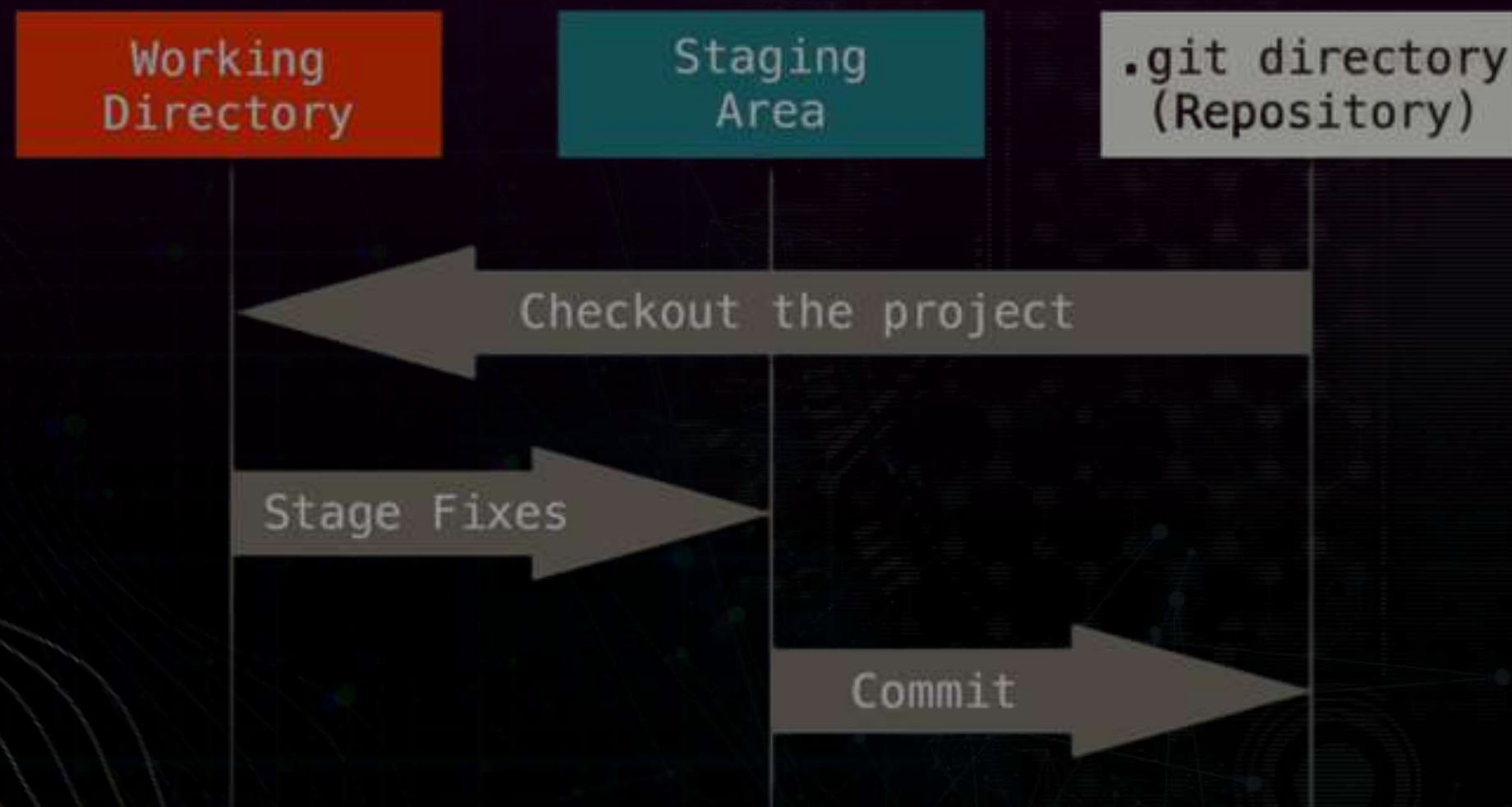
GIT FLOW



DOSW COMPANY



PROYECTO GIT SECCIONES PRINCIPALES



Es una copia de una version del proyecto, estos archivos se sacan de la base de datos comprimida en el directorio de git y se colocan en disco para que los puedan usar o modificar.



Es un archivo generalmente contenido en el directorio de git que almacena la informacion acerca de lo que va en la proxima confirmacion, a veces se le denomina indice



Es donde se almacena los metadatos y la base de datos de objetos para el proyecto, es la parte mas importante de git



COMANDOS GIT

GIT INIT

Crea un nuevo repositorio Git o reinicia uno existente, añade una carpeta oculta .git en el directorio donde almacena los metadatos, historial de versiones y registros del proyecto

GIT CLONE COPIED_URL_LINK

Copia un repositorio existente, incluyendo registros y versiones, desde servidores remotos como GitHub o GitLab a tu máquina local.

GIT STATUS

Muestra detalles sobre archivos modificados, archivos no rastreados y archivos preparados y listos para ser enviados

GIT ADD . Y GIT ADD FILE_NAME

Añade tus cambios al area de preparacion, indica a git que el repositorio debe actualizar estos cambios una vez el usuario ejecute el comando

GIT COMMIT -m "COMMIT_MESSAGE"

Guarda los cambios que has realizado en el repositorio local, una vez ejecutado crea una instancia del repositorio en ese momento

GIT PUSH

Sincroniza tu repositorio remoto con el repositorio local, una vez ejecutado el repositorio remoto reflejara todos los cambios que hayas confirmado localmente

GIT CHECKOUT BRANCH_NAME

Cambia desde la rama donde estes parado a la rama branch_name

GIT CHECKOUT -B BRANCH_NAME

Crea una nueva rama llamada "branch_name" desde la rama donde estes parado y cambia a la rama creada



COMANDOS GIT

GIT PULL

Recupera y fusiona los cambios del repositorio remoto con los del repositorio local, combina git fetch y git merge

GIT MERGE <NOMBRE_RAMA>

Permite realizar una fusion e integracion de los cambios de la rama nombre_rama a la rama en la que estes parado

GIT REBASE

Es otra forma de integrar cambios entre ramas, pero funciona de forma distinta a git merge. Fusionar conserva el historial completo tanto de la rama principal como de la rama de características. Rebase, por otro lado, sobrescribe el historial de confirmaciones para hacerlo lineal y limpio.

GIT LOG

Muestra todo el historial de confirmaciones de tu rama actual. Incluye cuatro campos clave: confirmación, autor, fecha y mensaje de confirmación

GIT LOG --ONELINE Y GIT LOG --STAT

La bandera --oneline causa en el git log que se muestre en pantalla: un commit por linea, los primeros 7 caracteres del SHA, mensaje del commit, la bandera --stat causa que muestre los archivos que se modificaron en cada confirmacion, numero de lineas añadidas o eliminadas y una linea resumen

GIT BRANCH

Muestra la lista de todas las ramas del repositorio

GIT BRANCH NOMBRE_RAMA

Crea una nueva rama llamada "nombre_rama"

GIT BRANCH -D NOMBRE_RAMA

Elimina la rama llamada "nombre_rama"



COMANDOS GIT

GIT DIFF

Muestra la diferencia entre tu directorio de trabajo actual y el area de preparacion

GIT DIFF COMMIT 1 COMMIT 2

Muestra la diferencia entre el commit 1 y el commit 2

GIT DIFF BRANCH1..BRANCH2

Compara las diferencias entre branch1 y branch 2

GIT STASH

Permite guardar temporalmente los cambios almacenados al directorio actual y restablece tu directorio de trabajo al ultimo estado comprometido

GIT STASH POP

Si has guardado algunos cambios en el pasado y quieres incluirlos en tu directorio de trabajo, esto aplicará los últimos cambios almacenados al directorio actual y los eliminará de la lista de cambios almacenados.

GIT CONFIG --GLOBAL USER.NAME "YOUR NAME"

Establece el nombre de usuario git para todos los repositorios de tu sistema en "your name"

GIT CONFIG --GLOBAL USER.EMAIL "EMAIL"

Establece la direccion de correo electronico apra todos los repositorios de tu sistema



COMANDOS GIT

GIT CONFIG -LIST

Muestra los ajustes git de todos los niveles: a nivel de sistema, a nivel global y a nivel local

GIT CLEAN

Elimina los archivos no rastreados del directorio de trabajo

GIT CLEAN -N

Muestra una lista de archivos y directorios no rastreados que se eliminarían sin borrarlos realmente. Esto te permite revisar los cambios antes de ejecutarlos.

GIT RESET --(SOFT,MIXED,HARD) HEAD~1

Permite deshacer cambios restableciendo el HEAD Actual a un estado anterior específico, para mover el HEAD a un commit anterior tiene tres variantes: suave que retrocede a una confirmación anterior pero con los cambios actuales, mixta que retrocede a una confirmación anterior y elimina los cambios de la confirmación actual y hard que apunta a una confirmación anterior y elimina las actualizaciones de la confirmación actual y del área de preparación.

GIT REMOTE ADD <REPO_NAME> <REMOTE_URL>

Crea una conexión entre tu repositorio git local y el repositorio git remoto, permitiéndote introducir y extraer cambios entre ellos, origen es por defecto

GIT FETCH

Te permite revisar los cambios en el repositorio remoto antes de fusionarlos en el local. Descarga los cambios y actualízalos en ramas de seguimiento remotas. Para los que no estén familiarizados, las ramas de seguimiento remoto son copias de ramas de repositorios remotos.



RECOMENDACIONES

Mantener el Flujo de Trabajo Limpio y estructurado y colaborativo

Realizar un seguimiento eficaz de los cambios a lo largo del tiempo, para posteriormente comprender que se modifico en cada uno

Mensajes claros que describan que se realizo, asi es mas sencillo saber que incluye cada cambio

Para cualquier nueva funcion o correccion, crea una rama trabaja en ella y fusionala, puede crear ramas desde el repositorio en github y actualizar en local.

Utiliza .gitignore que contiene archivos y directorios relacionados con el proyecto que no es necesario rastrear



DOSW COMPANY

ESTRUCTURAS DE DATOS

Importantes estructuras de datos que se utilizan para almacenar una serie de elementos y procesarlos

ARRAY VS ARRAY LIST

- Tamaño Fijo que debe especificarse y no puede modificarse posteriormente
- Solo puede contener tipos primitivos (int, double, boolean, etc)
- Tiene acceso a elementos muy rapido
- No soporta genericos, puede ser menos seguro ya que es mas facil introducir errores de tipo
- Acceso mediante [] y longitud con .length
- Puede ser unidimensional o multidimensional

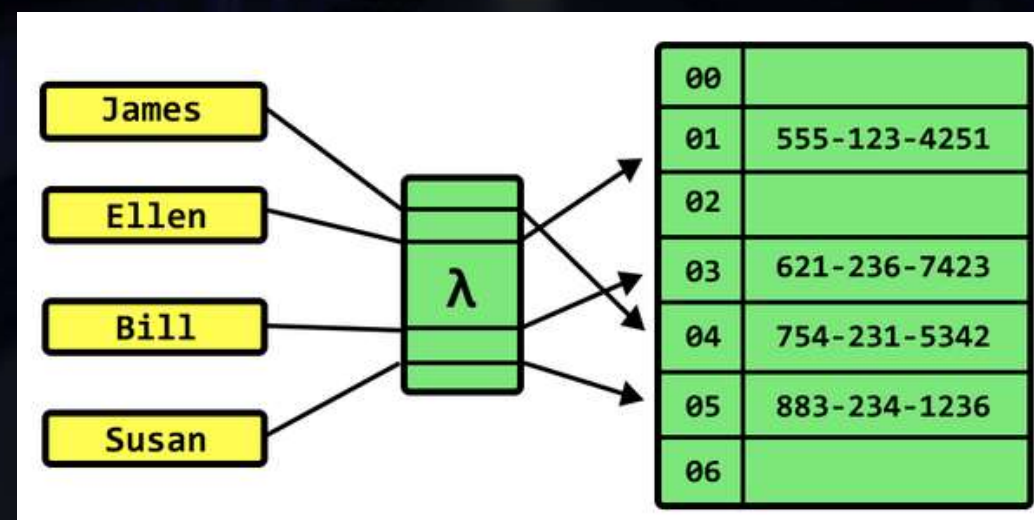
```
public class Array {  
    public static void main(String args[]) {  
        int a[]=new int[5];  
        a[0]=20;  
        a[1]=30;  
        a[2]=40;  
        a[3]=50;  
        a[4]=60;  
  
        for (int i=0; i<a.length;i++)  
            System.out.println(a[i]);  
    }  
}
```

- Tamaño dinamico, se expande o se contrae automaticamente segun los elementos que contenga
- Solo puede almacenar objetos, para usar los tipos primitivos Java los convierte mediante autoboxing (int a Integer)
- Es mas lento a comparacion en especial cuando necesita redimensionarse
- Soporta genericos mejorando la seguridad de tipos y evitando errores en tiempo de ejecucion
- add(), remove(), size(), indexOf(), contains()
- Es solo unidimensional

```
public class Array {  
    public static void main(String args[]) {  
        ArrayList a= new ArrayList();  
  
        a.add("Motorola");  
        a.add("Samsung");  
        a.add("Oneplus");  
        a.add("Apple");  
  
        Iterator itr =a.iterator();  
  
        while (itr.hasNext())  
        {  
            System.out.println(itr.next());  
        }  
    }  
}
```

HASHMAP VS HASHTABLE

- Un HashMap en Java es una implementación de la interfaz Map que almacena pares clave-valor.
- Usa internamente la implementación de hash table
- Las claves son únicas, pero los valores pueden repetirse.
- Internamente usa una tabla hash para ubicar rápidamente los elementos.
- Es no sincronizado → más rápido en entornos de un solo hilo.
- Permite una clave null y varios valores null.
- Un Hashtable también almacena pares clave-valor y usa una tabla hash internamente.
- Las claves son únicas, valores pueden repetirse.
- Es sincronizado → seguro para multihilo, pero más lento.
- No permite claves null ni valores null.
- Es una clase legacy (heredada de Java 1.0), anterior al HashMap. Hoy en día casi siempre se prefiere HashMap o ConcurrentHashMap.





HASHSET VS TREESSET

- Es una colección en Java que almacena elementos únicos (sin duplicados).
- Internamente usa una tabla hash para guardar los datos.
- No mantiene ningún orden en cómo se guardan o muestran los elementos.
- Permite un solo elemento null.
- Es muy rápido para buscar, agregar o eliminar elementos (en promedio $O(1)$).

```
HashSet<String> frutas = new HashSet<>();  
frutas.add("Manzana");  
frutas.add("Pera");  
frutas.add("Manzana"); // No se agrega porque ya existe  
System.out.println(frutas); // Orden aleatorio
```

- También es una colección que almacena elementos únicos.
- Internamente usa un árbol rojo-negro (estructura de árbol balanceado).
- Mantiene los elementos ordenados automáticamente (por orden natural o usando un Comparator).
- No permite null en Java moderno.
- Es más lento que HashSet ($O(\log n)$) porque mantiene el orden mientras agrega/busca/elimina.

```
TreeSet<String> frutas = new TreeSet<>();  
frutas.add("Manzana");  
frutas.add("Pera");  
frutas.add("Banana");  
System.out.println(frutas); // [Banana, Manzana, Pera] ordenado
```



STRING VS STRING BUILDER VS STRING BUFFER

- Es inmutable → una vez creado, su valor no se puede cambiar.
- Cada vez que modificas un String, en realidad se crea un nuevo objeto en memoria.
- Más seguro para uso en múltiples hilos porque no cambia, pero menos eficiente si hay muchas concatenaciones.

```
String texto = "Hola";  
texto += " Mundo"; // Se crea un nuevo objeto "Hola Mundo"
```

- Es mutable → puedes cambiar el contenido sin crear nuevos objetos.
- No está sincronizado → más rápido, pero no es thread-safe (no usar en múltiples hilos sin control).
- Ideal cuando necesitas modificar cadenas muchas veces en un solo hilo.

```
StringBuilder sb = new StringBuilder("Hola");  
sb.append(" Mundo"); // Modifica el mismo objeto  
System.out.println(sb); // Hola Mundo
```

- Es igual que StringBuilder pero sincronizado → es thread-safe.
- Más lento que StringBuilder debido al costo de sincronización.
- Se usa cuando varios hilos necesitan modificar la misma cadena.

```
StringBuffer sbf = new StringBuffer("Hola");  
sbf.append(" Mundo"); // Modifica el mismo objeto de forma segura  
System.out.println(sbf); // Hola Mundo
```




DOSW COMPANY

PROGRAMACION FUNCIONAL

Es un paradigma de programación que trata la computación como la evaluación de funciones matemáticas y evita cambios de estado y datos mutables. Se centra en aplicar y componer funciones para transformar datos, en lugar de modificar el estado del programa.



CONCEPTOS CLAVE

FUNCIONES PURAS

Funciones que, dado el mismo input, siempre devuelven el mismo output y no tienen efectos secundarios (no modifican variables externas).

INMUTABILIDAD

Los datos no se pueden modificar después de ser creados, lo que ayuda a evitar errores y facilita el razonamiento sobre el código.

FUNCIONES DE ORDEN SUPERIOR

Funciones que pueden tomar otras funciones como argumentos o devolver funciones como resultado

COMPOSICION DE FUNCIONES

Combinar varias funciones para crear una nueva función más compleja, pasando el resultado de una función como entrada a la siguiente.

ENFOQUE DECLARATIVO

Se describe qué se quiere lograr, no cómo se debe lograr, evitando detalles de implementación y estructuras de control como bucles



DOSW COMPANY

VENTAJAS

CODIGO MAS
LIMPIO



MAYOR FACILIDAD
PARA PRUEBAS



MAYOR
MANTENIBILIDAD



MAYOR PARALELISMO Y
CONCURRENCIA





EJEMPLO

NUMEROS MAYORES A 10 EN LA
LISTA

CON CICLOS Y CONDICIONALES

```
List<Integer> numeros = List.of(11, 8, 9, 15, 39, 1, 4, 83);
```

```
List<Integer> numeros = List.of(11, 8, 9, 15, 39, 1, 4, 83);

int contador = 0;

for (int numero: numeros) {
    if(nuemero > 10){
        contador ++;
    }
}

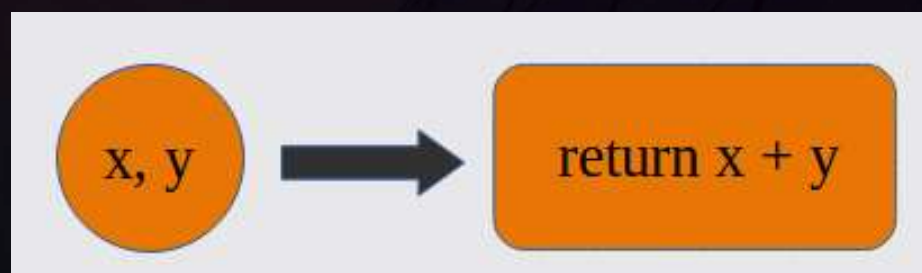
System.out.println(contador);
```

CON UN ENFOQUE FUNCIONAL Y
LAMBDAS

```
Long resultado = numeros.stream().filter(num -> num > 10).count();
System.out.println(resultado);
```


EXPRESION LAMBDA

Esta caracterizada por la siguiente sintaxis



Una expresión lambda representa el método abstracto de una interfaz funcional, una interfaz funcional es aquella que solo tiene un método abstracto, puede tener cualquier cantidad de métodos default o estáticos, pero solamente puede tener un método abstracto. Este método puede ser representado por una expresión lambda.

```
int resultado = numeros.stream()
    .filter(n -> n % 2 == 0)           // Filtrar pares
    .map(n -> n * 2)                   // Multiplicar por 2
    .reduce(0, (acum, n) -> acum + n); // Sumar todos

System.out.println("Suma de pares * 2: " + resultado);

// Ejemplo extra: Mostrar cada número procesado
numeros.stream()
    .filter(n -> n > 3)                 // Mayores que 3
    .map(n -> n * n)                   // Elevar al cuadrado
    .forEach(System.out::println);    // Imprimir cada uno
```

EXPRESION LAMBDA

OPERACIONES DE CREACION

Son la que inician el flujo de datos

- `stream()` → Convierte una colección en un stream.

```
java
List<Integer> lista = Arrays.asList(1, 2, 3);
Stream<Integer> flujo = lista.stream();
```

✦ Usada en todos los retos.

- `Stream.of(...)` → Crea un stream a partir de valores fijos.

```
java
Stream<String> flujo = Stream.of("A", "B", "C");
```

- `Arrays.stream(array)` → Convierte un arreglo en stream.

```
java
int[] numeros = {1, 2, 3};
IntStream flujo = Arrays.stream(numeros);
```

OPERACIONES INTERMEDIAS

- Transforman o filtran el flujo.
- Son lazy (perezosas): no se ejecutan hasta que llega una operación terminal.
- Devuelven otro Stream.
- Puedes encadenar multiples operaciones intermedias

OPERACIONES TERMINALES

- Cierran el stream.
- Devuelven un resultado no-stream (colección, valor, nada).
- Una vez que usas una operación terminal, no puedes reutilizar el mismo stream.
- Solo puedes usar una terminal para cerrar

[Creación]	→	[Intermedias]	→	[Terminal]
<code>.stream()</code>		<code>.filter()</code>		<code>.collect()</code>
		<code>.map()</code>		<code>.reduce()</code>
		<code>.sorted()</code>		<code>.forEach()</code>

EXPRESION LAMBDA CON STREAM

Filter Filtra los elementos segun la condicion

```
List<String> frutas = Arrays.asList("Manzana", "Pera", "Banana");

List<String> frutasConA = frutas.stream()
    .filter(f -> f.contains("a"))
    .collect(Collectors.toList());

System.out.println(frutasConA);
```

collect Es una operacion terminal de un stream, sirve para recoger los elementos procesados y guardarlos en una coleccion o estructura

```
List<String> lista = stream.collect(Collectors.toList());
Set<String> conjunto = stream.collect(Collectors.toSet());
Map<String, Integer> mapa = stream.collect(Collectors.toMap(k -> k, v -> v.length()));
```

peek sirve para ver o espiar lo que esta pasando en cada paso de la tuberia del stream sin modificar los datos

```
List<String> resultado = estudiantes.stream()
    .map(e -> e.getNombre().toUpperCase()) // transforma
    .peek(nombre -> System.out.println("Procesando: " + nombre))
    .collect(Collectors.toList()); // recoge en lista
```

ForEach h Recorre todos los elementos de una coleccion

```
List<String> frutas = Arrays.asList("Manzana", "Pera", "Banana");

frutas.forEach(fruta -> System.out.println(fruta));
```

Map p Transforma cada elemento de la coleccion en otro valor

```
List<String> frutas = Arrays.asList("manzana", "pera", "banana");

List<String> frutasMayus = frutas.stream()
    .map(f -> f.toUpperCase())
    .collect(Collectors.toList());

System.out.println(frutasMayus);
```

reduce e Combina todos los elementos en un solo resultado

```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5);

int suma = numeros.stream()
    .reduce(0, (acum, n) -> acum + n);

System.out.println("Suma: " + suma);
```



EXPRESION LAMBDA CON STREAM

sorted

Ordena el flujo de datos

```
List<Integer> numeros = Arrays.asList(5, 1, 3, 2, 4);

List<Integer> ordenAsc = numeros.stream()
    .sorted() // Ascendente
    .toList();

List<Integer> ordenDesc = numeros.stream()
    .sorted(Comparator.reverseOrder()) // Descendente
    .toList();
```

min(Comparator) /
max(Comparator)

```
List<Integer> numeros = Arrays.asList(10, 4, 7, 1, 9);

int minimo = numeros.stream()
    .min(Integer::compare)
    .get();
```

Devuelve el elemento minimo o maximo

```
List<Integer> numeros = Arrays.asList(10, 4, 7, 1, 9);

int maximo = numeros.stream()
    .max(Integer::compare)
    .get();
```

distinct

Elimina duplicados

```
List<Integer> repetidos = Arrays.asList(1, 2, 2, 3, 3, 3);

List<Integer> sinDuplicados = repetidos.stream()
    .distinct()
    .toList();
```

skip

Salta los primeros n elementos

```
List<String> nombres = Arrays.asList("Ana", "Luis", "Pedro", "Maria");

List<String> sinPrimeros2 = nombres.stream()
    .skip(2)
    .toList();

System.out.println(sinPrimeros2); // [Pedro, Maria]
```

any Match/ all Match/ none Match

validan si algun/todos/ninguno cumplen una condicion

limit

Toma solo los primeros n elementos

```
List<String> nombres = Arrays.asList("Ana", "Luis", "Pedro", "Maria");

List<String> primeros2 = nombres.stream()
    .limit(2)
    .toList();

System.out.println(primeros2); // [Ana, Luis]
```

```
boolean hayPar = numeros.stream()
    .anyMatch(n -> n % 2 == 0);
```

```
boolean todosPositivos = numeros.stream()
    .allMatch(n -> n > 0);
```

```
boolean ningunoNegativo = numeros.stream()
    .noneMatch(n -> n < 0);
```


NOS VEMOS LA PROXIMA SESION

FASE 2 ONBOARDING - PRUEBA TECNICA

Para la siguiente clase:

Traigan los comandos de Git con su funcion - Pueden traerlos impresos o escritos

Realicen el taller de programacion funcional - Fecha limite el dia Martes 4 de Agosto

Traigan computador para la sesion teorica



ANDRES.CANTOR-U@ESCUELAING.EDU.CO